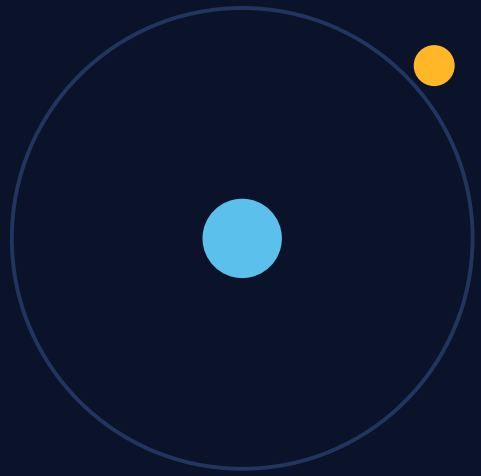
DATABASE MANAGEMENT SYSTEMS · J-COMPONENT PROJECT · REVIEW 1

OrbitalTrace

A Relational Database for Tracking Satellite Image Reconstruction Pipelines



BATCH

Batch 9

GUIDE

Dr. SUDHAKAR K

STAGE

Review 1

TEAM MEMBERS

Kapilan R
25BCE0537

Nithish S
25BCE0518

Tharun Prasath M
25BCE0506

Saiprasath S
25BCE0539

Presented for the DBMS J-Component · Vellore Institute of Technology

Abstract

Satellite sensors capture raw frames that are often blurry, partial, or cloud-obscured. External reconstruction algorithms clean them into usable images, often over several refinement attempts. Today this process is tracked in scattered logs and file names, so lineage, parameters and quality results are hard to trace.

OrbitalTrace is a relational database that records the pipeline around the reconstruction algorithm: satellites and sensors, raw captures, reconstruction jobs and their parameters, output images, quality scores, and refinement history. It does not process images; it stores only pipeline metadata.

The design uses ER / EER modelling, mapping rules to a relational schema, and integrity constraints. A working prototype shows the lineage graph and per-pixel provenance of each output image.

Keywords

Satellite imagery · Data lineage · Relational model · ER / EER · Quality tracking

Core idea

Lab notebook, not laboratory: track the process, not the reconstruction.

Outcome

A normalized schema and SQL-backed lineage queries.

Introduction

Without a structured record, nobody can reliably answer basic accountability questions about a reconstructed satellite image.

Which frames?

Which raw frames fed a given output image

Who and how?

Which analyst ran the job and with what parameters

How good?

What quality score the result achieved

First or refined?

Whether an image is a first attempt or a refinement

What we are building

A relational audit trail around a black-box reconstruction algorithm.

What we are not building

The reconstruction algorithm itself. No AI, image processing or computer vision.

Existing Systems Reviewed

Satellite Data Portals

USGS EarthExplorer, Copernicus Data Space

Catalog raw captures for download but do not model the downstream reconstruction pipeline or refinement lineage.

ML Experiment Trackers

MLflow, DVC, Weights & Biases

Track generic runs and parameters well, but are not modelled for satellite sensors, orbital metadata or multi-frame contribution weighting.

Processing Logs / Flat Files

What most pipelines use today

Plain text logs or file names encoding job info. No enforced keys, no relational queries, no integrity guarantees.

Ground-Segment Software

Agency / institution proprietary

Built for mission operations, not as an open, queryable lineage-and-quality database.

Research Gap

The gap: no lightweight, relationally-modelled system tracks raw-frame-to-reconstruction lineage, job parameters, quality scores and refinement history together, with integrity enforced.

Capability	Portals	ML trackers	Flat logs	OrbitalTrace
Raw-frame to image lineage	No	Partial	No	**Yes**
Satellite / sensor metadata	Yes	No	No	**Yes**
Multi-frame contribution weight	No	No	No	**Yes**
Refinement history	No	Partial	No	**Yes**
Enforced keys and integrity	Partial	No	No	**Yes**

Proposed System Architecture

Data Sources

Satellite and sensor details, raw frame captures, analyst inputs

→

Black-Box Reconstruction

Multi-frame fusion with cloud masking. Job metadata and pixel provenance are recorded

→

OrbitalTrace Database

Relational schema with PK / FK, constraints and referential integrity

→

Query Layer

SQL queries for lineage, quality trends and refinement ancestry

→

Interface (later)

Web dashboard with lineage graph and pixel provenance viewer

Data flow: raw frames feed a reconstruction job, the job produces an image, the image receives quality assessments, and a refined image points back to the image it came from. The database stores each of these links.

List of Modules

1. Satellite & Sensor Management
2. Raw Frame Capture Management
3. Reconstruction Job Management
4. Image & Pixel Provenance
5. Quality Assessment
6. Analyst, Mission & Query

Module 1 · Satellite & Sensor Management

Techniques & methodology

- ER to relational mapping: strong entity to table, weak entity to table plus owner key
- EER specialization: Sensor splits into Optical, Infrared and Radar (disjoint, total) with one table per class
- Calibration history kept as a weak entity so every record belongs to one sensor
- Keys and foreign keys enforce that no sensor exists without a satellite

Schema structures

```
SATELLITE(Satellite_ID, Name, Orbit_Type,
Launch_Date, Agency, Mission_ID)

SENSOR(Sensor_ID, Resolution, Sensor_Type,
Satellite_ID)

CALIBRATION_RECORD(Sensor_ID, Calibration_Date,
Offset_Parameters)

OPTICAL / INFRARED / RADAR_SENSOR(Sensor_ID,
subclass attribute)
```

ER relationships: CARRIES (Satellite 1:N Sensor), CALIBRATED_BY (Sensor 1:N Calibration_Record, identifying), PART_OF (Satellite N:1 Mission).

Module 2 · Raw Frame Capture Management

Techniques & methodology

- Each capture stored with timestamp, cloud cover percentage and processing status
- Composite attribute Orbit_Position flattened into Latitude, Longitude and Altitude
- Domain checks keep cloud cover between 0 and 100
- Total participation: every frame must come from a registered sensor

Schema structures

```
RAW_FRAME(Frame_ID, Capture_Timestamp, Latitude,
Longitude, Altitude, Cloud_Cover_Pct, Status,
Sensor_ID)

PK: Frame_ID

FK: Sensor_ID references SENSOR
```

ER relationships: CAPTURED_BY (Sensor 1:N Raw_Frame), total on the Raw_Frame side.

Module 3 · Reconstruction Job Management

Techniques & methodology

- The reconstruction itself is an external multi-frame fusion with cloud masking; OrbitalTrace records its metadata
- The M:N link between frames and jobs becomes a separate table with a combined primary key
- Contribution_Weight stored on each link as a relationship attribute
- Job parameters, algorithm, timing and status logged for every run

Schema structures

```
RECONSTRUCTION_JOB(Job_ID, Algorithm_Used,
Parameters, Start_Time, End_Time, Status,
Analyst_ID)

CONTRIBUTES_TO(Frame_ID, Job_ID,
Contribution_Weight)

PK of link: (Frame_ID, Job_ID)

FKs: Analyst_ID, Frame_ID, Job_ID
```

ER relationships: CONTRIBUTES_TO (Raw_Frame M:N Reconstruction_Job), RUN_BY (Job N:1 Analyst).

Module 4 · Image & Pixel Provenance

Techniques & methodology

- Every output image links to the job that produced it
- Pixel provenance layers: class map (observed or synthesized), uncertainty (0 to 1) and source index (dominant source frame)
- Pixels with no source frame are filled by interpolation and flagged as synthesized with uncertainty 1.0
- Refinement lineage by a recursive foreign key, traced with a recursive SQL query

Schema structures

```
RECONSTRUCTED_IMAGE(Image_ID, Resolution,
Output_Format, Generated_Timestamp, Job_ID,
Original_Image_ID)

FK: Job_ID references RECONSTRUCTION_JOB

FK: Original_Image_ID references
RECONSTRUCTED_IMAGE (NULL for a first attempt)
```

ER relationships: PRODUCES (Job 1:N Image), REFINED_FROM (Image 1:N Image, recursive).

Module 5 · Quality Assessment

Techniques & methodology

- Several metrics can score one image, so each assessment is a weak entity
- Partial key Metric_Type plus the owner Image_ID forms the primary key
- Scores stored with the assessor, so quality trends can be compared across refinements
- Aggregate SQL (average, max per job) for quality reports

Schema structures

```
QUALITY_ASSESSMENT(Image_ID, Metric_Type, Score, Assessed_By)

PK: (Image_ID, Metric_Type)

FK: Image_ID references RECONSTRUCTED_IMAGE
```

ER relationships: ASSESSED_BY (Reconstructed_Image 1:N Quality_Assessment, identifying), total on both sides.

Module 6 · Analyst, Mission & Query

Techniques & methodology

- Analyst and mission tables give every job an owner and every satellite a mission
- 1:1 relationship mapped by placing the foreign key on Mission
- Join queries list contributing frames per job and the settings used
- Recursive CTE returns the refinement ancestry of any image

Schema structures

```
ANALYST(Analyst_ID, Name, Role)

MISSION(Mission_ID, Name, Objective, Start_Date,
Lead_Analyst_ID)

FK: Lead_Analyst_ID references ANALYST
```

ER relationships: LEADS (Mission 1:1 Analyst), RUN_BY (Job N:1 Analyst), PART_OF (Satellite N:1 Mission).

Dataset Used

Input frames

Multi-frame satellite-style terrain imagery with cloud cover. Each frame has a capture time, cloud percentage and status.

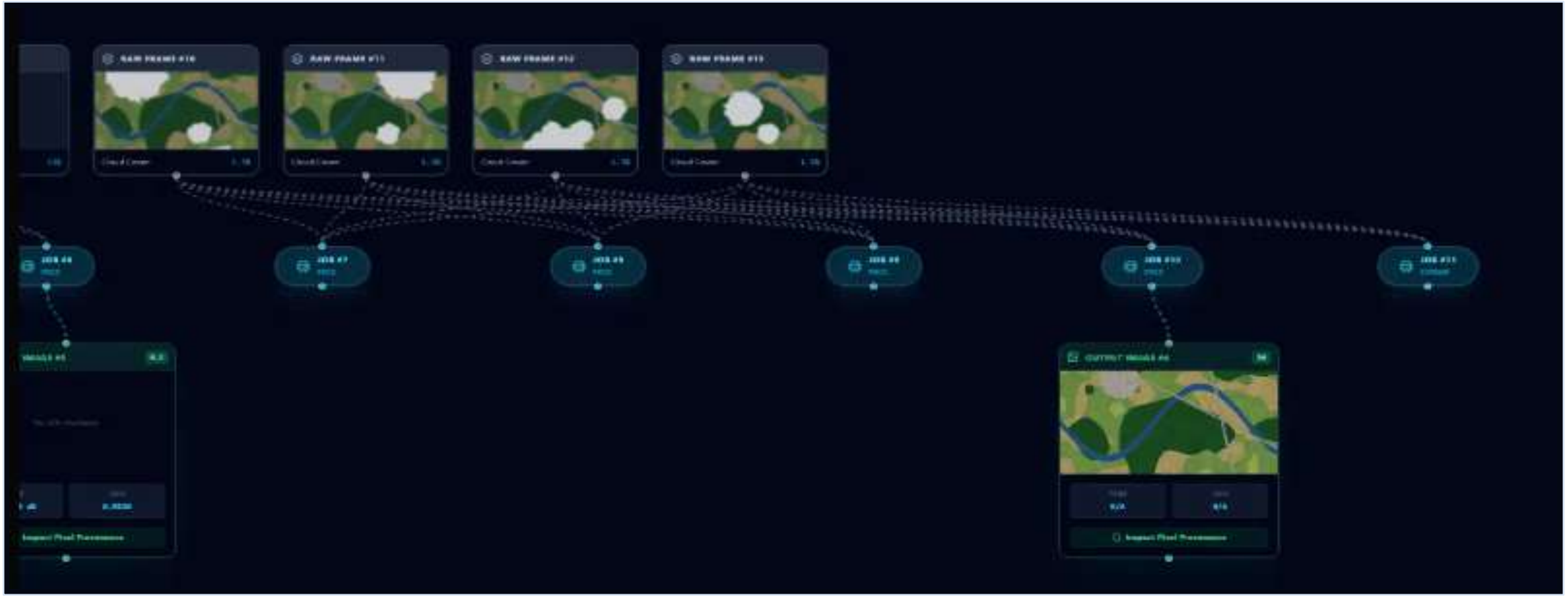
Metadata

Satellite, sensor, calibration, analyst and mission records created as sample data for the schema.

Note

The screenshots use generated imagery with cloud blobs. Replace this with the public dataset or source you actually used, if any.

Lineage Graph: Frames → Jobs → Output Images



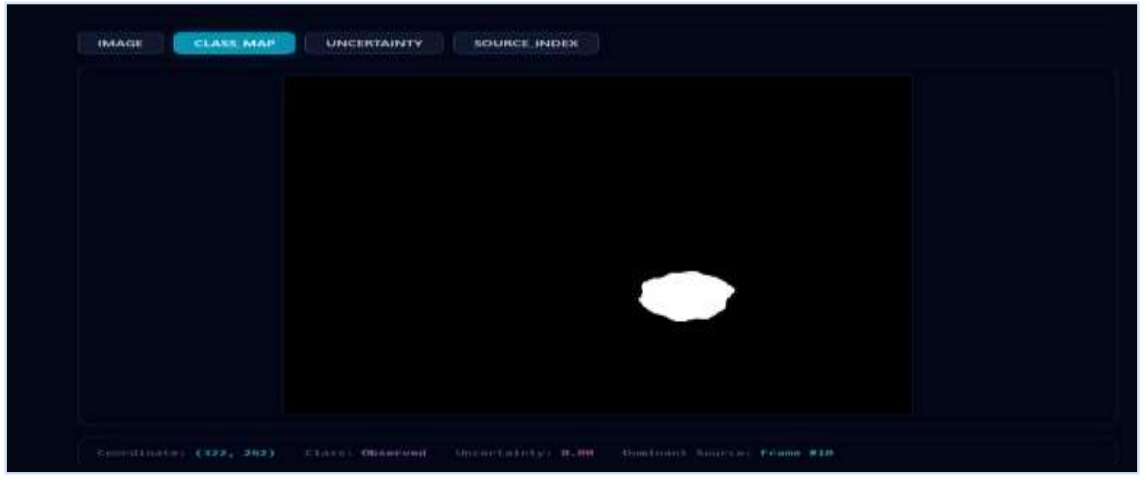
Raw frames feed reconstruction jobs, which produce output images with pixel provenance inspection

Each raw frame links to the jobs it contributed to, and each job links to its output image.

Pixel Provenance Viewer



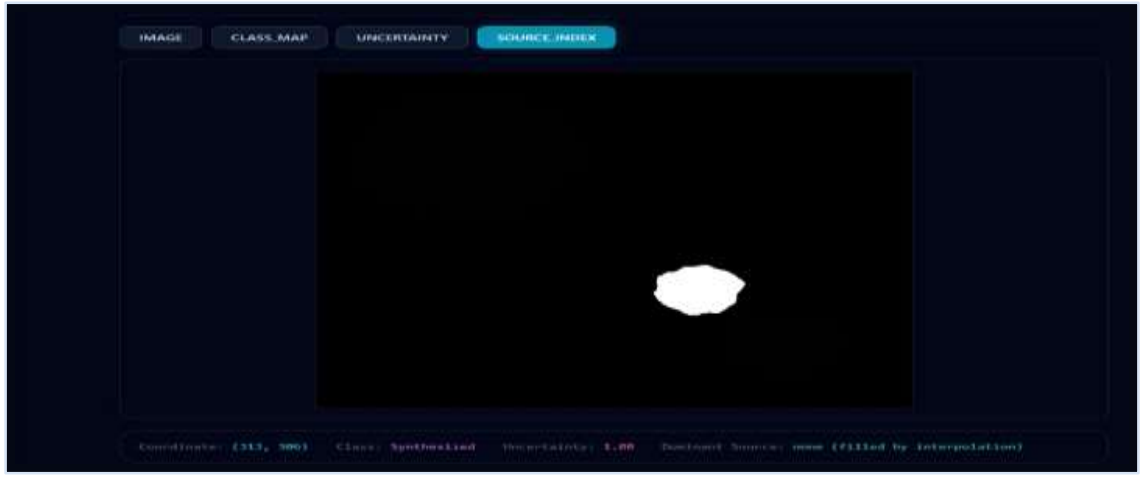
IMAGE: reconstructed output



CLASS_MAP: observed vs synthesized pixels



UNCERTAINTY: per-pixel uncertainty



SOURCE_INDEX: dominant source frame, white = none

References

1 **Elmasri, R., & Navathe, S. B.**
Fundamentals of Database Systems. Pearson. ER / EER modelling and relational mapping chapters.

2 **Database Systems course lecture notes**
DBMS J-Component, Vellore Institute of Technology, including conversion of ER model to relational model.

3 **USGS EarthExplorer**
Earth observation data portal. earthexplorer.usgs.gov

4 **Copernicus Data Space Ecosystem**
Sentinel satellite data portal. dataspace.copernicus.eu

5 **MLflow and DVC documentation**
Experiment tracking and data versioning tools. mlflow.org, dvc.org